Implementation Plan

Phase 1: Basic CLI Setup

Goal: Create the command-line tool structure.

Build:

architex
architex parse
architex audit
architex ask
architex generate

Use:

Python
Typer
Rich

Outcome:

> User can run Architex from terminal and see clean CLI output.

Phase 2: Code Parser

Goal: Read the repo and understand files.

Build `parser.py`.

It should scan Python files and extract:

file path
lines of code
imports
local dependencies

Example output:

checkout.py imports payment_router.py
refund.py imports payment_router.py
invoice.py imports payment_router.py

Outcome:

> Architex knows which files exist and which files depend on each other.

Phase 3: Repository Knowledge Graph

Goal: Build the first version of the knowledge graph.

Build `graph_builder.py`.

Graph structure:

Node = file
Edge = import relationship

Example:

checkout.py ⟶ payment_router.py
refund.py ⟶ payment_router.py
invoice.py ⟶ payment_router.py

Outcome:

> Architex can identify load-bearing files.

Your MVP already planned this using NetworkX, where nodes are file paths and edges are imports.

Phase 4: Risk Scoring

Goal: Rank files by danger level.

Build `risk_engine.py`.

Simple MVP formula:

Risk Score = Incoming Dependencies × Lines of Code

Example:

payment_router.py
Imported by 17 files
495 lines of code

Risk Score = 17 × 495 = 8415

Outcome:

> Architex can say, “This file is risky because many files depend on it.”

Phase 5: Documentation Scanner

Goal: Check if risky files are documented.

Build `doc_scanner.py`.

Scan:

/docs
/adr
README.md
*.md files

Check whether high-risk files are mentioned in docs.

Outcome:

> Architex can find files that are both important and undocumented.

This matches the core MVP idea: scan complex, load-bearing code and find missing documentation gaps.

Phase 6: `architex audit`

Goal: Show the main MVP result.

Command:

architex audit

Output:

Top Undocumented Risk Files

1. payment_router.py
  Imported by: 17 files
  LOC: 495
  Risk Score: 8415

Docs: Missing

2. auth_session.py
Imported by: 12 files
LOC: 420
Risk Score: 5040
Docs: Missing

Outcome:

> This is the first demo-worthy command.

Phase 7: Risk Q&A

Goal: Let user ask why a file is risky.

Build `ask.py`.

Command:

architex ask payment_router.py "why is this risky?"

The answer should use:

risk score
incoming dependencies
outgoing dependencies
documentation status
source code summary

Outcome:

> Architex becomes more than a scanner. It explains risk.

Phase 8: ADR Generator

Goal: Generate missing architecture documentation.

Build `generator.py`.

Command:

architex generate payment_router.py

Output:

docs/payment_router_ADR.md

The ADR should include:

Context
Inferred Decision
Systemic Role
Trade-offs
Risks
Recommended Human Review

Outcome:

> User gets a useful document, not just a warning.

Your original plan already includes this ADR generation step using Claude and saving the result into `/docs`.

Phase 9: Simple HTML Report

Goal: Make the demo attractive visually.

Build `report.py`.

Command:

architex report

Output:

architex-report.html

Report should show:

Total files scanned
Top undocumented risk files
Highest-risk file
ADR coverage
Simple dependency map
Recommended next actions

Outcome:

> Good for demos, judges, founders, and investors.

Phase 10: Polish

Goal: Make it feel like a real developer tool.

Add:

Rich tables
progress spinners
clear error messages
green success panels
red/yellow risk labels
sample repo for demo
README

Final demo commands:

```
architex audit
architex ask payment_router.py "why is this risky?"
architex generate payment_router.py
architex report
```

Simple Build Order

Build in this exact order:

1. CLI shell
2. Parser
3. Graph
4. Risk score
5. Docs scanner
6. Audit command
7. Ask command
8. Generate ADR command
9. HTML report
10. Polish demo